

Jacob Barnhill, Marquise Douglas,
Malloy Hawkins, Karma McCoy, Michael Poe
CSC 4033 - 001
Group 3
10/9/2026

Team Contract

How your team works together

1. **Communicating.** Which channels will you use (chat, email, shared documents, in person), and which kinds of messages go on which channel? How quickly must members respond? (Discussion 01, Q3)
 - a. Discord
 - b. Mon-Fri: 24 hrs to respond
 - c. Sat/Sun: Optional
2. **Settling disagreements.** How will you tell what kind of disagreement you are having? Who decides, and how, when you cannot agree? (Discussion 01, Q5)
 - a. Group Discussion (Vote)
 - b. RNG(Rock-Paper-Scissors) in the event that the vote is tied
3. **Missed expectations.** What will the team do when a member does not follow one of the rules in this contract? (Discussion 01, Q2)
 - a. A reasonable deduction of points from peer review*

How code joins the codebase

1. **Getting a change in.** List the steps a change must go through before it becomes part of your shared code. Name the point at which the change stops belonging only to its author. (Discussion 03, Q1)
 - a. Everyone works in a static pairing system, checking one another's code. When the code is planned to be pushed to a repository everyone checks the code to make sure there are no conflicts between pairs.
2. **Review authority.** Who can stop a change, and on what grounds? (Discussion 03, Q2)
 - a. Anyone can stop a change on the grounds that it degrades the overall code or causes conflict with another person's contribution
3. **Author-reviewer disagreements.** When do an author and a reviewer settle a disagreement themselves, and when does it go to your rule in item 2? (Discussion 03, Q3)
 - a. Disagreements are settled among the group members
4. **The main branch.** What must always be true of your main branch? Name the branching strategy you will use, and say why it fits a team of your size and how often you will release. (Discussion 04, Q1).
 - a. *What must always be true of your main branch?*
 - i. It must reflect a production ready state.
 - b. *Name the branching strategy you will use, and say why it fits a team of your size and how often you will release.*

- i. A recursive merge is when two branches have independent changes and need to be merged. This fits the team of our size because requiring everyone to merge at once would be a hindrance logistically. We will release when needed and will consult one another on whether or not it is ok.
- 5. **Time on a branch.** How long may a piece of work stay on its own branch before it joins the main branch? What happens to work that runs past that limit? (Discussion 04, Q2)
 - a. *How long may a piece of work stay on its own branch before it joins the main branch?*
 - i. For our group a piece of finished work can stay in its own branch up until a new piece of finished work from another member is attempting to join the main branch.
 - b. *What happens to work that runs past that limit?*
 - i. Work that runs past the limit will be ignored, the new piece of work will be reviewed and then merged.
- 6. **Joining the main branch.** When work joins the main branch, will your history show that it was done on a separate branch? When, if ever, may a member rebase? (Discussion 04, Q3)
 - a. *When work joins the main branch, will your history show that it was done on a separate branch?*
 - i. Yes
 - b. *When, if ever, may a member rebase?*
 - i. A rebase might be considered assuming the majority of the group agrees to it for whatever reason, be it for the sake of clear documentation or as a project requirement. However it is unlikely that it would come to that.
- 7. **Version numbers.** What will each part of your service's version number tell the people who depend on it? Who decides the number for a release? What else will tell them what changed, that the number cannot? (Discussion 04, Q4–Q5)
 - a. *What will each part of your service's version number tell the people who depend on it?*
 - i. The X.Y.Z format (Major.Minor.Patch) will detail the following information:
 - 1. X: MAJOR version when you make incompatible API changes
 - 2. Y: MINOR version when you add functionality in a backward compatible manner
 - 3. Z: PATCH version when you make backward compatible bug fixes
 - b. *Who decides the number for a release?*
 - i. Decisions pertaining to the entire project typically are left up to majority rule within the group
 - c. *What else will tell them what changed, that the number cannot?*
 - i. Updated documentation.

Hypothesis

1. **Roles on a task.** For every task, who is responsible for doing it, and who is accountable for approving it? May one person hold both roles on the same task? Will you also record who is consulted and who is kept informed?
 - a. Each task will be a pseudo partnership with one primary contributor and a backup in case the primary contributor is out of commission or can't pull their weight. The group acts as the approving party and those who are consulted or review the code will be named in the program. One person may hold.
2. **Way of working.** Rate this project against the conditions from Discussion 02, Q5. Name the way of working you would choose on your own and defend it. If you would combine more than one, say which parts come from which
 - a. Agile method due to the focus on collective accountability, small team size, and allowance of each member to manage their own responsibilities.
3. **Assigning tasks.** Who decides which person works on which task?
 - a. Group delegation based on experience with the particular tasks
4. **Meetings.** List the recurring meetings you would hold. For each one, give its purpose, how often it happens and how long it lasts. Leave out any meeting that is not worth its time.
 - a. A meeting will be held every Wednesday
 - b. Purpose: pushing changes to the git repository and checking code
 - c. Duration: The meeting lasts as long as necessary to review code
5. **Pair-written code.** If two or more people wrote a change together, does it still need a separate reviewer?
 - a. Yes, the rest of the group members will act as separate reviewers